

Metrics That Write Themselves: Evolving an Evaluator from Its Own Blind Spots

Xing Zhang Yanwei Cui Guanghui Wang Zhihao Lin Peiyang He*
AWS Generative AI Innovation Center

Abstract

Agents improve quickly against a reliable automatic metric and stall without one, and the applications that need them most, report generation among them, are the ones nobody knows how to score. Can the metric write itself? Saying what makes an answer good is hard; pointing at something wrong with one is easier, so the metric we evolve is a pool of small Python *operators* that each flag a candidate for one named defect, or abstain, and vote. Asking a model for operators directly does not work: 183 candidates realise only 96 distinct behaviours, from one narrow region of an enormous space. EvalCEGAR instead borrows counterexample-guided abstraction refinement from program verification. It reads the pool as an abstraction and searches for a *collision*, two answers the operators score identically, one correct and one not. That pair, not a prompt, is the authoring request, and when a collision defeats every attempt the loop widens what an operator may read rather than resampling. On MBPP+ and HumanEval+, a sandbox whose hidden unit tests give exact ground truth, the loop writes a 55-line operator that closes 15.4% of the gap between flagging nothing and a perfect filter on 428 unseen tasks ($+0.0065$, $p=0.0010$) at a quarter of our best hand-written operator’s flags. On the benchmark it never saw it matches that operator’s effect exactly on a third of the flags. Six of eight runs admit such an operator and all six help out of sample; our 15 hand-written operators applied together as one filter *lose* accuracy. An LLM judge on the same information ties that delta on a nearly disjoint set of candidates, and charges a model call per candidate forever where the operator charges none.

1 Introduction

A self-improving system needs a metric before it needs a policy. Given a reliable automatic metric, LLM agents improve rapidly against it: self-refinement, self-debugging and reflective loops all assume a signal separating better answers from worse [13, 20, 5]. Without one, progress stalls, and the substitutes are weak: reference overlap misses semantics by construction [16, 10], and an LLM judge carries position, verbosity and self-preference biases that belong to the judge, not the answer [24, 21]. This bites hardest on open-ended output, a report or a plan, where there is neither a reference answer nor a settled account of what a good one looks like.

This paper asks whether the *metric* can be evolved automatically. What we evolve is not a score but a pool of *operators*: small Python functions that each read a task and one candidate answer and either *flag* it for one specific defect, pass it as *clean*, or *abstain*. A pool becomes a metric by voting: a candidate is rejected when enough members flag it. This form follows an asymmetry that survives the absence of a reference: saying how good an answer is needs a standard nobody has written down; pointing at something wrong with it usually does not. Each operator states one such objection, so it can be read, run and falsified on its own, and the pool’s validity is a measured property rather than an assumed one: every member had to earn its place on a downstream decision before it joined.

*Corresponding author: peiyan@amazon.com

What our first runs showed. We first ran the obvious loop: ask the model for an operator, keep it if a gate accepts it, repeat. It stalls in two ways, and we take both as the problem statement. *Obstacle 1: the space of operators is enormous, and a model asked for “an evaluation operator” samples a tiny, fixed region of it.* Resampling does not move that region: scoring all 295 operators the study authored at that narrow interface yields no discovery at all (Section 5.2). *Obstacle 2: what it does write repeats itself behaviourally, and most of it is rejected.* Five runs produced 183 operators that flag anything, realising only 96 distinct flag sets (Section 6). A rejection teaches nothing either: the loop learns that a candidate failed, not which distinction it failed to draw.

Our approach. We borrow a discipline from program verification: counterexample-guided abstraction refinement sharpens an abstraction only when a concrete counterexample proves the current one too coarse [7]. The operator pool is that abstraction, mapping each candidate answer to a *signature*, the vector of verdicts the pool returns for it. Two answers with the same signature are indistinguishable to the metric, so a pair sharing one while disagreeing on ground truth proves the metric cannot express the distinction. EvalCEGAR hands that pair to the model as the specification. Each admission re-partitions the signatures, so the next specification differs by construction: the target, not the sampler, supplies the diversity pressure (Obstacle 2). When a pair defeats every attempt, the loop widens the *interface* an operator may read rather than sampling again, since a distinction the interface cannot see is unreachable at any sample count (Obstacle 1). An operator is admitted only if it improves the decision the metric is deployed to make, not if it covers more known faults, and admitted operators vote.

Why we validate on code. The motivation is domains *without* a usable metric, yet every number here comes from a domain that has an exact one: Python problems from MBPP+ and HumanEval+ [1, 4, 11], whose hidden unit-test suites settle correctness outright. A method that writes metrics cannot be evaluated where the truth is unknown, so we build it where the truth is known and withheld. The operators are domain-specific by construction, so a new domain inherits the loop, not this pool. What transfers is narrow but checkable: the operators never read the oracle, four screens enforce that they do not reconstruct one (Section A), and oracle labels enter only on the training split, at a price a new domain can pay: of our 75 training tasks only 10 carry both a correct and an incorrect candidate that survive the visible checks, and that was enough to admit operators that transfer (Section B).

Contributions.

- **EvalCEGAR.** A loop that evolves an evaluation metric from its own blind spots, the authoring request being a counterexample it finds for itself (Section 4).
- **Operators that hold up out of sample.** Six of eight runs admit an operator and all six help on 428 unseen tasks; the best is 55 lines of Python and matches our best hand-written operator on a second benchmark at a third of its flags, where a model judge on the same information ties it while flagging a near-disjoint set (Section 5.1).
- **What each mechanism is worth.** Ablated inside the loop on identical information: the admission question alone moves admissions from zero to one, and the narrow interface admits nothing in 336 attempts (Section 5.2).
- **Composition solved exactly, not searched.** Scoring all 2.3M subsets the loop can compose ranks the objective it used to combine them as low as the 5th percentile of that space, and an untuned class-weighted one at the 91st or above (Section 5.3).

2 Related work

Automatic evaluation of open-ended output. Both substitutes above have been sharpened without changing shape: learned metrics replace overlap counting with a model’s own similarity judgement [23], and judge frameworks hand the judge explicit criteria to fill in [12], refined by decomposing coarse ones and filtering the redundant [19]. The output is still a single scalar whose parts a reader cannot separate. EvalCEGAR produces a different object, a pool of small, executable, individually falsifiable *objections*. A pool can gain or lose one without retraining, which is what makes a metric of this form evolvable. We compare against both kinds: the judge ties us on the endpoint while dissenting on which candidates to drop (Section 5.1), so the claim is about how a checkable metric is produced, not about beating a scalar.

LLM-driven program search and CEGAR. FunSearch evolves programs against a *given* evaluator [18], and automated agent design searches agent code against a *given* benchmark [9]. Ours is the complement: with the evaluator itself as the artefact there is no fixed fitness function, so the design problem moves from how candidates are mutated to what a candidate must prove before it is kept. Closest in output are methods that induce executable checks with an LLM and combine them, by a compact set of Python verifiers whose joint satisfaction approximates a labelled objective [17] or an unweighted vote over model-written yes/no questions [8]. Those search for checks that predict a label; we search for the counterexample showing the current ones cannot, and admit on the deployed decision instead. We run that recipe as a baseline (Section 5.2). CEGAR is a verification technique [7]; to our knowledge its transfer to *metric* synthesis is new (Section 4).

Diversity and test generation. Quality-diversity search keeps a population from collapsing by maintaining an explicit archive of behaviours [15]; our diversity pressure comes instead from re-partitioning the target after each admission, and Section 6 measures what that does not buy. Nor is this test generation: a property-based tester needs a property to falsify [6] and differential testing needs a second implementation [14], and our setting supplies neither.

3 Preliminaries

Tasks and data. Every task is a Python programming problem from MBPP+ or HumanEval+ as distributed by EvalPlus [11], whose hidden test suite is the exact oracle. It labels the training samples the loop targets and admits on, and nothing else reads it but the scoring script. A statement also shows a few example calls with their expected results, which the solver sees too; we call these the *visible checks*, and they are not held-out information. We drew 150 MBPP+ tasks, split 75 to run the loop on and 75 held out, and added the 228 remaining MBPP+ tasks and 125 of the 164 HumanEval+ tasks to the held-out side.² Results are reported on those 428 unseen tasks as *MBPP+ held-out* (75), *MBPP+ additional* (228) and *HumanEval+* (125), the only group from a different task distribution.

Selection accuracy. For each task t we sample many model solutions and let $V(t)$ be those that pass the visible checks, duplicates included: the candidates a deployed system would plausibly accept, so the faults that matter are the ones surviving them. A metric must choose among them. It drops from $V(t)$ what it flags, leaving the kept set $K(t)$, and one sample is drawn uniformly from $K(t)$, or from $V(t)$ if the metric flagged everything. *Selection accuracy* is the probability that this sample is correct, averaged over tasks; Δ always means its change against the metric that flags nothing.

Decidability and headroom. A task is *decidable* only if $V(t)$ holds both a correct and an incorrect sample; on any other task the drawn sample’s correctness is settled whatever the metric does. Only 58 of the 428 held-out tasks are decidable at our sampling depth, so selection accuracy over all 428 runs from 0.7170 when nothing is flagged up to 0.7593 for a perfect operator. Absolute Δ values are small for that reason, so we also report each as a fraction of that reachable range, the *headroom*. The endpoint is also *recall-dominated*: a flagged correct sample merely leaves the kept set, while a caught incorrect one changes what remains to draw from.

What we compare against. Before building the loop we wrote 15 operators by hand, ten static checks on the candidate’s syntax tree and five that run it. They are the pool the loop starts from and, applied together as one filter, our human baseline. Our strongest single hand-written operator, the *comparator* throughout, flags a candidate whose behaviour disagrees with the plurality of the other samples for its task. The second baseline is an LLM judge, asked one question per candidate under exactly a level-1 operator’s information, in four configurations over two models and two prompts (Section I).

The model that writes operators. One frozen model (Claude Opus 4.7, no fine-tuning and no gradient update anywhere in this work) emits operator source code, 47–48 calls per run under a 60-call ceiling across 8 rounds. The faulty candidates its operators are judged on come from weaker models (Llama 3.1 8B Instruct and Claude Haiku 4.5, with Amazon Nova Micro and Mistral 7B

²HumanEval+ prints no asserts, so its visible checks are synthesised from the docstring examples; we keep the 125 tasks that yield at least one such check the reference solution passes.

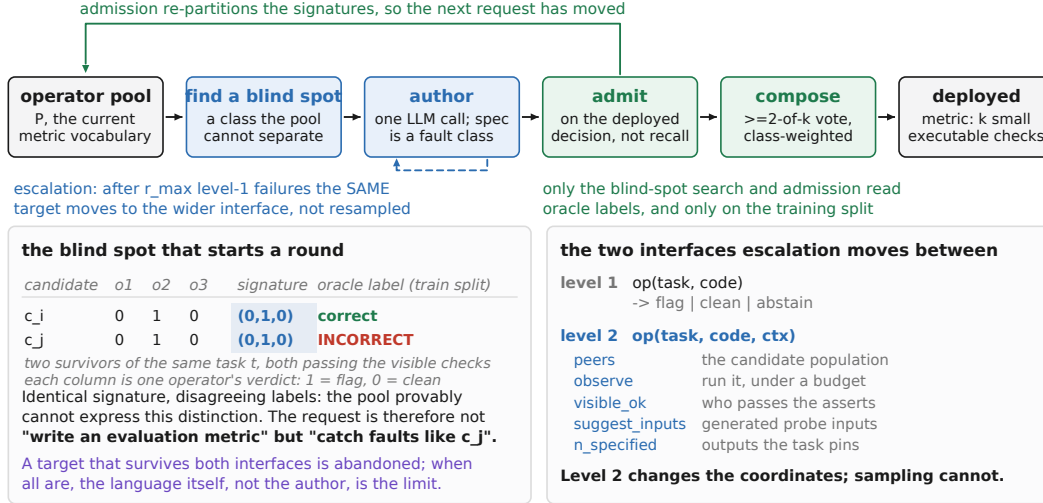


Figure 1: EvalCEGAR. The authoring request is a *blind spot*: answers the pool maps to one signature while ground truth disagrees (left). Admission re-partitions the signatures, so the next request has moved, and a blind spot that resists level 1 escalates (right) rather than being resampled.

Instruct added for the deeper draw of Section 6), so the model that writes bad code never wrote the operators that judge it. What EvalCEGAR ships, the operator pool and its vote, is pure Python with *no* inference-time model call.

Keeping the loop honest. Four screens reject an authored operator before the gate sees it, each closing a way an earlier round of this work reconstructed the oracle by accident: deciding from the candidate’s syntax without running it, keying on a constant the fault generator happens to emit, branching on prompt keywords, and reaching a flag by text match while executing the candidate elsewhere. None fires on any of the 15 hand-written operators, so they are not a ban on ordinary detectors (Section A). Every Δ is tested against a size-matched shuffle null: per task the operator’s own flag count is redrawn from that task’s own samples, so the null fixes how much it flags and randomises only where the flags land. That licenses “this rule beats flagging nothing” but not “rule A beats rule B ”, the claim Section 6 reports on, and multiple comparisons carry a Bonferroni correction within a family of related tests and a Benjamini–Hochberg one across a screen of many [2]. And *registered* means the prediction was written down before the script that tests it existed.

4 Method

In one sentence. Find two candidate answers the *current* pool scores identically, one correct and one incorrect; ask the model for an operator that catches faults of that kind; widen what an operator may read when the request proves unreachable; and admit on whether the deployed decision improves. Figure 1 is the loop and Algorithm 1 one round of it.

Collision targeting. A *collision*, or blind spot, is a signature class holding both correct and incorrect candidates, and the largest one in the pool becomes the next target. The colliding pair locates the request but does not state it: the request covers every training sample that fails in the same way, because a specification naming two samples is satisfiable by a lookup table (Section A). A target that survives every attempt at both interfaces is abandoned, and once every target has been abandoned the loop returns an *insufficiency certificate* for the operator vocabulary: the metric *language*, not the author, is the limit. That is what makes a failed run informative: it names the distinction the vocabulary could not express.

Interface escalation. Level 1 is $op(task, code)$. When $r_{\max}=3$ level-1 attempts fail on a target, the *same* target escalates to level 2, $op(task, code, ctx)$, where ctx lets the operator see the candidate’s *peers* and run any of them on inputs of its own choosing, under a hard budget of 600

Algorithm 1 EvalCEGAR, one round on one target. Δ_D is the change in selection accuracy on the training tasks D . Ground-truth labels y are read on the training split only, at lines 2 and 10; AUTHOR sees the specification and the interface, never y .

Require: operator pool P , training tasks D with candidate sets $V(t)$ and labels y , level-1 retry budget $r_{\max}$, total attempt budget R

```

1:  $\sigma_P(c) \leftarrow (o(t, c))_{o \in P}$  for every  $c \in V(t), t \in D$  ▷ the pool is the abstraction
2:  $\mathcal{C} \leftarrow \{ \text{signature classes containing both a } y=1 \text{ and a } y=0 \text{ candidate} \}$  ▷ the blind spots
3: if  $\mathcal{C} = \emptyset$  then return  $P$  ▷ the pool already separates every labelled pair
4:  $\kappa \leftarrow \arg \max_{\text{class} \in \mathcal{C}} |\text{class}|$  ▷ the target
5:  $\text{spec} \leftarrow$  every incorrect training sample that fails as  $\kappa$ 's do, and every correct one ▷ not just the pair
6:  $\ell \leftarrow 1$  ▷ interface level:  $\text{op}(\text{task}, \text{code})$ 
7: for  $r = 1 \dots R$  do
8:    $o \leftarrow \text{AUTHOR}(\text{spec}, \ell)$  ▷ one LLM call; a stated request, not "write a metric"
9:   if  $o$  fails a leakage screen, does not shrink  $\kappa$ , or is a function of  $\sigma_P$  then continue
10:  if  $\Delta_D(P \cup \{o\}) > 0 \wedge \text{helped} \geq 3 \wedge \text{helped} > \text{hurt}$  then
11:    return  $P \cup \{o\}$  ▷ admission re-partitions  $\sigma_P$ , so the next  $\kappa$  has moved
12:  if  $r = r_{\max}$  then
13:     $\ell \leftarrow 2$  ▷ same  $\kappa$ , wider interface  $\text{op}(\text{task}, \text{code}, \text{ctx})$ 
14: return  $P$  ▷  $\kappa$  abandoned: unreachable at either interface

```

observations per operator call over at most 16 peers (Figure 1, right, and Section A). This changes the coordinates of the search instead of resampling within them: if the useful operator is not expressible at level 1, no amount of sampling finds it.

Admission on the deployed decision. The final gate is a floor on the *decision*, evaluated on the training split: $\Delta > 0$, at least 3 tasks helped, helped $>$ hurt. It is *not* a floor on fault-class recall, which matters more than it sounds: a sparse, precise operator fails a recall floor by construction and is exactly what the endpoint wants. The recall-gated arm rejected our best operator for covering none of the 140 known faults the pool still missed (Section D).

Composition by error independence. Admitted operators are composed by a vote: a candidate is flagged when at least m of the k operators in a chosen subset S flag it, with $m=2$ and $k = |S| \leq K_{\max}=6$ unless stated otherwise. The subset is the one maximising an objective computed on the labelled training samples alone, with no development split, no tuned threshold and no tie-break, and unit weights rather than learned ones, which are hard to beat when the positive class is scarce [8]. The objective was originally $J = \text{TP} - \text{FP}$ over *corroborated* flags, those on which the vote fires; Section 5.3 shows J works *against* deployment and replaces it with $\text{TP} - \lambda \text{FP}$ at $\lambda = n_{\text{TP}}/n_{\text{FP}}$ read off the training split.

5 Results and analysis

Three questions: does the metric work on unseen tasks (Section 5.1), what does each mechanism contribute when it is removed (Section 5.2), and how good the composition step is once its whole search space can be scored exactly (Section 5.3).

5.1 Main result

Table 1 compares single operators. The loop's is 55 lines of Python and reaches 94.2% of the comparator's effect with a quarter of its flags. Its +0.0065 rises to +0.0481 over the 58 decidable tasks alone (Table 2), and its flags overlap the closest hand-written operator at Jaccard 0.245, so it is no rediscovery. What it does reads in one sentence: keep the peers that pass the prompt-visible asserts, run them and the candidate on perturbed inputs, and flag disagreement with the surviving plurality. Every admitted level-2 operator is a variant of that algorithm, since the loop rewards a new distinction rather than a new way to draw one (Section 6). The same table holds the contrast that motivates the loop: in a domain people find *easy*, the 15 hand-written operators as one filter *lose* accuracy.

Table 1: The authored operator against the strongest hand-written one and an LLM judge, on the 428 held-out tasks and their 2592 samples. % of headroom is Δ as a fraction of the 0.0424 a perfect operator could add (Section 3); *calls* is model requests per candidate scored. The three positive rows lie within 0.0005 of one another on Δ : what separates them is flags and cost.

	Δ	% of headroom	p	flags	calls
hand-written comparator	+0.0069	16.3	0.0130	515	0
all 15 hand-written operators as one filter	-0.0067	-15.8	n/a	1425	0
LLM judge, best of four configurations	+0.0070	16.5	0.0010	158	1
loop-authored operator	+0.0065	15.4	0.0010	126	0

Table 2: Per-pool transfer of the authored operator, each against its own shuffle null ($b=1000$ resamples). HumanEval+ differs from the authoring pool in oracle module, prompt style and difficulty, and nothing was re-tuned. +/− counts tasks helped and hurt; false alarms is the rate at which *correct* samples are flagged, authored / hand-written. Per-pool headroom: 0.0392, 0.0429, 0.0433.

pool	tasks	decidable	Δ	p	z	+/−	false alarms
MBPP+ held-out	75	6	+0.0047	0.0400	1.74	+2/−1	0.054 / 0.168
MBPP+ additional	228	34	+0.0038	0.0020	3.38	+8/−4	0.041 / 0.129
HumanEval+	125	18	+0.0125	0.0010	6.28	+10/−1	0.027 / 0.071

A model judge on the same information. Asked one question per candidate under a level-1 operator’s exact interface, and scored as a metric on the identical endpoint and null, the best of four judge configurations reaches +0.0070 (Table 1), so we do not claim to beat it. It is neither the same verdict nor the same cost. The two flag sets meet at Jaccard 0.105, 27 flags of 257; each side’s exclusive flags are null alone (+0.0010 at $p=0.068$, and -0.0007); and the 27 they agree on are worth +13/−1 tasks, with the union above both parts at +0.0073. A checkable operator and a model judgement fail on different inputs, so their agreement is a high-precision region neither reaches alone. Cost separates them permanently: 47 authoring calls and nothing per candidate, against one call per candidate forever, 1762 for a single pass here. Two asymmetries belong with that comparison: the author model judges at +0.0040, below what it authors, and both benchmarks are public, so a judge may have memorised solutions where an operator cannot (Section I).

A pool, not one operator. Eight runs differ only in the model’s sampling, so their spread is EvalCEGAR’s own sampling distribution: 6 of 8 admit an operator, all 6 are positive out of sample (median +0.0029, range +0.0009 to +0.0065, 4 of 6 individually significant), all 6 are at level 2, and all four fitting screens are clean on all 6. Across the whole study the gate has admitted 26 operators, the smaller of the two pools Section 5.3 composes over: on the held-out tasks 25 are non-harmful and 17 are individually significant against their own nulls, where chance predicts 1.3, at a median of +0.0036. The 15 hand-written operators are not such a pool: 9 flag anything at all, 5 are non-harmful, and 1 is significant. Admission reads the training split alone, so this is transfer.

Transfer to an unseen benchmark. Two facts stand out (Table 2). Of 13 rules measured on all three pools, 12 loop-authored operators and the comparator, this one alone is positive *and* clears its own per-pool null on all three, where the comparator reaches $p < 0.05$ on HumanEval+ alone. And on HumanEval+, the pool from a different task distribution, it matches the comparator exactly and beats it on parsimony: the same 10 helped tasks, the same single hurt task and the same +0.0125, which is 28.8% of that pool’s headroom, with 36 flags against 103. Each pool was first checked to hold enough decidable tasks for a perfect operator to register, so a zero would mean absence, not no power. Transfer also holds along a second axis, to 9320 candidates from three generators the composed rule never saw, at 22.1% of that draw’s headroom against 18.9% here (Section G).

5.2 Mechanism ablations

Removing either mechanism stops admission outright (Table 3): each is necessary, not merely helpful. The narrow-only arm’s failure is not a budget artefact, because level 1’s own ceiling is below chance: scoring all 295 level-1 operators the study authored, on the pooled 503 tasks with *no split and no*

Table 3: The two mechanism ablations, each run inside the loop on its own candidates, over the same three seeds and the same training-split labels, with only the named component changed. *Calls* is the model’s authoring calls per seed, so neither ablation is a starved run. No level-1 attempt was ever admitted, in either arm or in the study’s 336 (Section D).

arm	what changes	calls	admissions
the loop (reference)	–	47 / 48 / 47	1 / 1 / 1
recall floor at the gate	the admission objective	48 / 48 / 48	0 / 0 / 0
level 1 only, no escalation	the interface	48 / 48 / 48	0 / 0 / 0

selection cost gives 9 at $p < 0.05$ against 14.75 expected by chance, so Benjamini–Hochberg returns *no discoveries* at a 10% false-discovery rate.

An independent search agrees on where that wall is. Commissioning checks against a balanced labelled sample and admitting them for *predicting the label*, as the induction methods of Section 2 do, admits 2 of 48 candidates at level 1, whose $+0.0016$ its own null does not reject, and 6 of 16 at level 2 for $+0.0054$ under a training-only combiner (Section I). Two searches sharing nothing but the interface both fail at level 1 and both work at level 2, which also bounds what the recall-floor row claims: an admission objective must not reward coverage, but predictive fit does not fail the way recall gating does.

The gate ledger. Recording *every* gate consultation locates that wall exactly. Admission needs three helped tasks; the narrow interface reaches two and stops there 55 times with no task hurt, and none of its 124 consultations reaches three, while the wider interface reaches three 6 times and every one was admitted (Figure 3). The extension does not buy the *direction*, which the narrow interface finds easily; it buys *generality*, the third distinct task. Targeting itself buys candidate viability rather than admissions: an operator abstains everywhere 34.5% of the time undirected against 2.1% directed (Section C).

5.3 Composition, enumerated

Composition chooses a subset of size at most 6 from the loop’s admitted operators, with no hand-written operator in either pool. Both pools collect the gate’s admissions across the study’s authoring arms and seeds, 26 operators and 35, the larger containing the smaller, and they realise 17 and 20 distinct flag behaviours, giving 313,911 and 2,007,327 subsets, small enough to *enumerate*. We scored every one on the held-out endpoint through the same code used everywhere else (Figure 2), so questions previously answered by inference become arithmetic and no p -value appears in this subsection. Two terms recur: a subset is *vacuous* when its vote flags nothing, and the *oracle argmax* is the best held-out Δ in the enumeration, an upper bound no training-data rule should reach. Every result spans four *cells*: each pool at each of two pinned settings of the peer set a level-2 operator reads.

The defect. J ranks its own argmax anywhere from the 4.9th to the 85.7th percentile of the space it searches across the four cells (C1 in Table 4), against an oracle argmax of roughly *twice* its delta ($+0.0300/+0.0314$ against J ’s $+0.0155/+0.0140$), and its within- k correlation with the endpoint is positive only at $k=2$, falling to -0.338 by $k=6$ (Figure 2c). Enumeration also shows the cause is *bias*, not *variance*: refitting on more labelled data converges (16–18 distinct selections at $n=210$ to 3–5 at $n=1195$), and it converges *below* the *frozen rule*, the subset pinned under an earlier rule of thumb. So freeze the selection rather than refitting it.

The repair. We registered six selection criteria before running, each computed from the training split alone ($n_{TP}=222$ true and $n_{FP}=1183$ false flags), ranked by the unchanged ≥ 2 -of- k rule and scored against the exact landscape (Table 4). Balanced accuracy is best of the six in all four cells, closing 39.8–90.2% of the gap to the oracle argmax, and it also beats label-prediction F_1 , the criterion the induction baseline admits on, in all four.

Why it works. Our registered prediction blamed aggregation, that J counts per *sample* while the endpoint averages per *task*; but C2, the endpoint’s own formula, never reaches the 85th percentile,

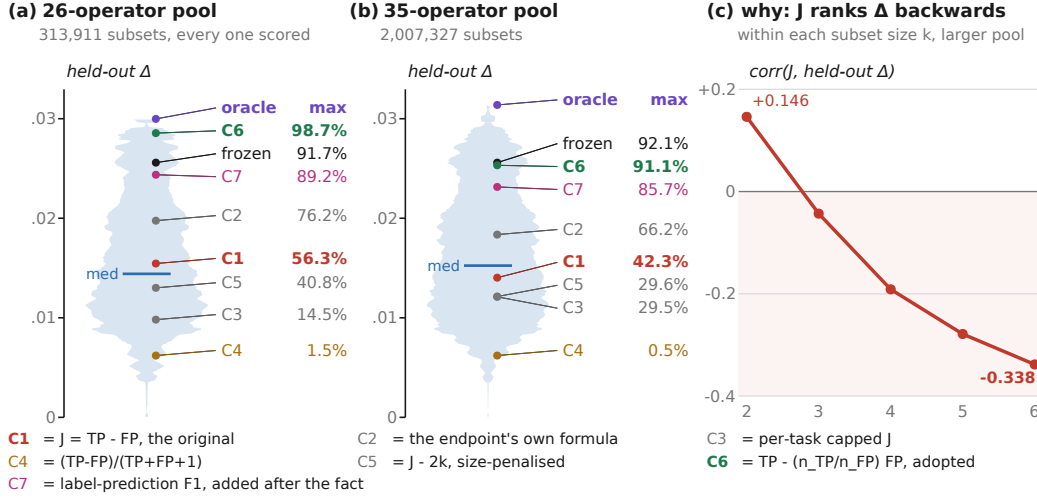


Figure 2: The composition search space, enumerated. (a,b) Held-out Δ of every ≥ 2 -of- k subset ($k \leq 6$) of the 26- and 35-operator pools, as a density, with C1–C7 of Table 4, the frozen rule and the oracle argmax each at its exact percentile. C7 was added after the fact. (c) Spearman correlation between J and the endpoint *within* each subset size on the larger pool: positive only at $k=2$, so the misalignment is no size artefact.

so that was not the defect. What matters is the class weight: balanced accuracy is monotone in $TP - 0.188 FP$, penalising a false alarm $5.33\times$ less than J 's 1:1, and C4 falls to the 1.5th percentile or below. That is the recall dominance of Section 3 resurfacing as a mis-specified objective, and it means *refitting is safe once the objective is right*, which a self-evolving library needs. Two registered checks hold: a split-half protocol re-selects C6 in 4 of 4 cells, and on a 12-point sweep HumanEval+'s optimal λ -plateau contains the MBPP training ratio in 4 of 4 cells while $\lambda=1$ never does, so the weight is a property of selection endpoints, not of those labels (Section F).

Safety over the whole closure. Of the 2,007,292 non-vacuous subsets of the larger pool, *zero* harm the endpoint (minimum $+0.0007$): admission makes composition safe over its *entire* combinatorial closure, exactly and with no power argument. The converse holds on the 15 hand-written operators, which admission accepts *none* of: the same combiner converges there to flagging nothing.

6 Negative results and limitations

Behavioural duplication. The pool repeats itself behaviourally rather than textually: no source-text pair among 187 candidates reaches Jaccard 0.8, yet the 183 that flag anything realise only 96 distinct flag sets (Figure 5). Neither cure worked. Targeting made it worse, the directed arm being the *least* diverse of three, and a novelty term at admission recovers a genuinely new operator in only 2 of the 11 candidates it blocks; handed the incumbent it duplicated, the model duplicates it again. So the restriction sits in the operator prior, not at the gate, which argues for an explicit behavioural archive [15].

Waste, not exhaustion. Nine of 14 admissions on three fresh seeds rediscovered an operator already selected, and a composition headline of $+0.0076$ on that pool did not survive the rediscovery screen we then added (Section H). Rarefaction over behavioural classes [3] still yields 2.00 new classes at the sixth seed: wasteful, not exhausted.

What the numbers do not show. Every Δ is measured against flagging nothing, never against another rule: no two rules separated in 9 paired tests [22], their differences living on 2–9 tasks, and the three loop operators that appeared to beat the comparator were rediscoveries of it. A level-2 verdict is also a function of the peer electorate: growing it threefold changes a verdict for all 35 operators, so their flags need a pinned electorate, though held-out Δ moves up rather than down (Section G). What admission discards is unmeasured (Section C). Both benchmarks come from one

distribution family, so an out-of-sample *task list* needs a third oracle port. And the loop is not the only route to an operator of this quality: a judge ties its delta, so what it buys is a complementary error profile, not a higher ceiling.

7 Conclusion

An evaluation metric can be authored automatically, and the artefact is small enough to read: 55 lines of Python, no weights, closing 15.4% of the distance to a perfect operator on 428 unseen tasks. It asks with a counterexample rather than a prompt, and three rules behind it carry over to any pool of executable checks:

- **Admit on the decision, not on coverage.** That change alone moved admissions from none to one per seed.
- **When nothing resolves a counterexample, widen the interface.** 336 attempts at the narrow interface admit none; the wider one admits six.
- **On a selection endpoint, weight the classes.** $TP - (n_{TP}/n_{FP})FP$, read off the training labels, reaches the top decile of a space whose precision-flavoured objective sits in the bottom 2%.

Where unit tests exist, run the unit tests. What remains is the setting that motivated the method, where an unresolved counterexample is the only evidence that a metric’s vocabulary, and so the claim it can support, has run out.

References

- [1] Jacob Austin, Augustus Odena, Maxwell Nye, Maarten Bosma, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.
- [2] Yoav Benjamini and Yosef Hochberg. Controlling the false discovery rate: a practical and powerful approach to multiple testing. *Journal of the Royal Statistical Society: Series B*, 57(1):289–300, 1995.
- [3] Anne Chao. Estimating the population size for capture-recapture data with unequal catchability. *Biometrics*, 43(4):783–791, 1987.
- [4] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.
- [5] Xinyun Chen, Maxwell Lin, Nathanael Schärli, and Denny Zhou. Teaching large language models to self-debug. In *International Conference on Learning Representations (ICLR)*, 2024.
- [6] Koen Claessen and John Hughes. QuickCheck: A lightweight tool for random testing of Haskell programs. In *International Conference on Functional Programming (ICFP)*, pages 268–279, 2000.
- [7] Edmund Clarke, Orna Grumberg, Somesh Jha, Yuan Lu, and Helmut Veith. Counterexample-guided abstraction refinement. In *Computer Aided Verification (CAV)*, pages 154–169, 2000.
- [8] Ben Griffin, Aaron Ontoyin Yin, Diego Vidaurre, Ugur Koyluoglu, et al. Random Rule Forest (RRF): Interpretable and manageable ensembles of LLM-generated questions for predicting success from unstructured data. *arXiv preprint arXiv:2505.24622*, 2025.
- [9] Shengran Hu, Cong Lu, and Jeff Clune. Automated design of agentic systems. *arXiv preprint arXiv:2408.08435*, 2024.
- [10] Chin-Yew Lin. ROUGE: A package for automatic evaluation of summaries. In *Text Summarization Branches Out*, pages 74–81, 2004.
- [11] Jiawei Liu, Chunqiu Steven Xia, Yuyao Wang, and Lingming Zhang. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.

- [12] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-Eval: NLG evaluation using GPT-4 with better human alignment. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.
- [13] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, et al. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [14] William M. McKeeman. Differential testing for software. *Digital Technical Journal*, 10(1):100–107, 1998.
- [15] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites. *arXiv preprint arXiv:1504.04909*, 2015.
- [16] Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. BLEU: a method for automatic evaluation of machine translation. In *Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 311–318, 2002.
- [17] Pouya Pezeshkpour and Estevam Hruschka. AutoPyVerifier: Learning compact executable verifiers for large language model outputs. *arXiv preprint arXiv:2604.22937*, 2026.
- [18] Bernardino Romera-Paredes, Mohammadamin Barekatain, Alexander Novikov, Matej Balog, et al. Mathematical discoveries from program search with large language models. *Nature*, 625:468–475, 2024.
- [19] William F. Shen, Xinchu Qiu, Chenxi Whitehouse, Lisa Alazraki, et al. Rethinking rubric generation for improving LLM judge and reward modeling for open-ended tasks. *arXiv preprint arXiv:2602.05125*, 2026.
- [20] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.
- [21] Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, et al. Large language models are not fair evaluators. *arXiv preprint arXiv:2305.17926*, 2023.
- [22] Frank Wilcoxon. Individual comparisons by ranking methods. *Biometrics Bulletin*, 1(6):80–83, 1945.
- [23] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. BERTScore: Evaluating text generation with BERT. In *International Conference on Learning Representations (ICLR)*, 2020.
- [24] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, et al. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS) Datasets and Benchmarks Track*, 2023.

A Request scope, screens and nulls

The request handed to the author covers a whole fault kind rather than the two colliding samples, because a two-sample request is satisfiable by a lookup table: an earlier round of this work measured exactly that, with operators firing on a handful of training faults and on none of a held-out class. The two kinds are faults that already fail the visible checks and faults that pass them and fail only the hidden suite, and the request covers whichever kind the colliding sample belongs to.

The subtlety of the four screens of Section 3 is the prompt-keyword one: an operator that branches on a keyword answers from the task’s identity rather than from the candidate in front of it, which is oracle reconstruction by dispatch table.

Level 2’s `ctx` exposes the candidate’s peers, a runner, the prompt-visible asserts and a generator of inputs, and nothing that reads the hidden suite. Inputs come from perturbing the arguments of the prompt-visible asserts, which the solver also saw, with the number of unperturbed ones exposed so

that an operator can tell the two apart. An operator that exceeds the observation cap abstains rather than stalling.

The shuffle null of Section 3 uses $b=1000$ resamples throughout, seeded so that a rerun reproduces them exactly.

B The transfer budget

Because oracle labels enter only on the training split, the cost of moving EvalCEGAR to an oracle-free domain is the cost of that split’s labels, and it must be quoted in *decidable* examples: an example informs the loop only if it carries both a good and a bad candidate that survive the visible checks. Of our 75 training tasks 10 are decidable, giving 80 (correct, wrong) pairs and four distinguishable levels of the training objective. That is demonstrably enough to *admit* operators that transfer (+0.0065) and demonstrably *not* enough to tune one scalar: a ≥ 4 -of-26 vote is worth +0.0087 held-out, and neither training Δ nor cross-validation inside training can distinguish it from the union.

Three measurements complete the budget. (i) The crossover for tuning aggregation is 20–30 decidable tasks: below it the choice collapses to the union in 9 of 9 directions; above it, rotating a held-back fold recovers +0.0077 ($p \leq 0.0004$, identical at $m=2, 3, 4$), 57% of the oracle’s gain. (ii) A better gate removes that requirement rather than paying it: 9 operators from three seeds of the target-only gate (Section C) score +0.0071 ($p=0.0022$) as a plain union with no threshold to set, so gate quality and aggregation are substitutes. (iii) Composition changes the currency altogether, and this is the cheapest line item: error independence needs *labelled flags*, not decidable tasks, and the same training split holds 426 of them. The ≥ 2 -of- k rule matches (i)’s rotated selection with 5 operators instead of 26 and nothing to tune, at a measured price of ~ 60 labelled candidates (at half that sample it degrades to +0.0065/+0.0033). So the budget line for a new domain is ~ 10 decidable examples to author and admit plus ~ 60 labelled candidates to decide which admitted operators vote, and the second number is the one a practitioner can actually produce, because it does not require a single example to hold both a correct and an incorrect candidate.

C Targeting versus accepting

An early ablation on a synthetic bank favoured the collision 4 admissions to 0, with 31 of 36 undirected candidates rejected as *untargeted*. That did not replicate on the natural bank and we discard the claim it licensed. A later single-flag experiment settles the division of labour instead: keeping the collision as the *target* but dropping it as the *acceptance test* raises distinct-metric yield from 0.38 to 1.67 per seed ($4.4\times$) with peak quality unchanged (+0.0063 vs +0.0065), and moves the binding rejection stage to the objective (60.3% of rejections, up from 28.6%). So the collision is doing two separable jobs, and only the first is worth its cost. What the gate discards is unmeasured in the other direction too: one operator it turns down for flagging nothing in training is our most reliable one held out.

D The mechanism ablations in full

Both arms run inside the loop on the loop’s own candidates, so neither is a re-scoring of operators authored under the other. Both read the same training-split labels, and no gate call in either arm hit its abort guard. The floor the recall-gated arm applies asks an operator to fire on at least 30% of the 140 fault classes the pool still misses, at no more than a 2% false-alarm rate on the samples that must pass. The narrow-only arm was also given double the level-1 attempts per target, 48 per seed against the full loop’s 24, and pooling its 144 with every level-1 attempt the eight full runs made gives the 336.

E The gate ledger in full

Of the 336 level-1 attempts of Table 3, the 124 that survived every earlier screen are the ones the gate saw and Figure 3 draws; a target is attempted at the narrow interface first, so most of an escalating run’s consultations are narrow ones and reading a run’s ledger as one interface would over-attribute them. Each of the six operators admitted across the eight runs has a recorded level-1 rejection immediately before it. The ledger doubles as the sensitivity analysis for the admission floor: at helped

≥ 2 the narrow interface would admit 55 of its 124 consultations, so the floor is what an interface has to clear, and what those admissions would transfer to is unmeasured.

no narrow-interface candidate ever passes two helped tasks; the wider interface does

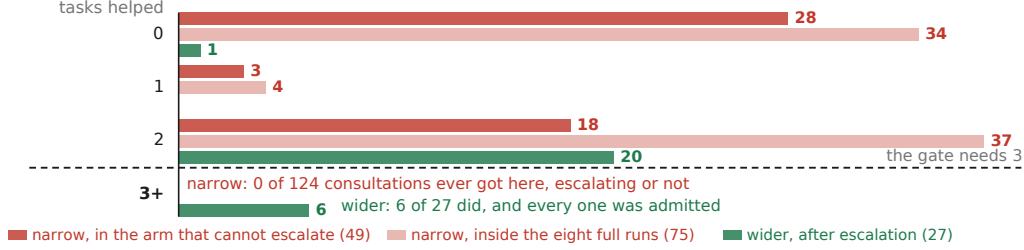


Figure 3: Every consultation of the admission gate, not just the ones that ended in an admission, by how many tasks the candidate helped on the training split the gate reads. Admission needs three, and only the wider interface ever gets there, whether or not the narrow run could escalate.

F The selection criterion: registration, robustness, and the class weight

All six criteria, their formulas and their predicted ranking were written down before the landscape was scored, so Table 4 reports where each choice lands rather than which choice was searched for.

Table 4: Six a-priori criteria, and one added afterwards, against the enumerated landscape of Figure 2. Cells run (26-operator pool, 35-operator pool) at each peer-set setting. C6 is the rule we adopt, and nothing is tuned: 0.1877 is the training split’s class ratio n_{TP}/n_{FP} . C7 is the induction baseline’s own admission signal and the only row not registered in advance.

criteria	criterion (training split only)	landscape percentile, 4 cells	held-out Δ
C1	$J = TP - FP$ (the original)	56.3 / 42.3 / 85.7 / 4.9	+0.0079 ... +0.0220
C2	the endpoint’s own formula on training data	76.2 / 66.2 / 78.3 / 82.1	+0.0184 ... +0.0209
C3	per-task capped J	14.5 / 29.5 / 50.4 / 44.1	+0.0098 ... +0.0138
C4	precision-flavoured $(TP - FP) / (TP + FP + 1)$	1.5 / 0.5 / 0.4 / 0.1	+0.0043 ... +0.0062
C5	size-penalised $J - 2k$	40.8 / 29.6 / 32.5 / 2.3	+0.0072 ... +0.0130
C6	balanced accuracy $\equiv TP - (n_{TP}/n_{FP}) FP$	98.7 / 91.1 / 92.7 / 94.1	+0.0236 ... +0.0286
C7	label-prediction F_1 (not registered)	89.2 / 85.7 / 85.0 / 85.8	+0.0218 ... +0.0244

Both registered robustness checks behind Section 5.3 are reported here in full. (i) *Split-half*. Choosing the criterion on half A of the held-out tasks and reporting on half B, with the split fixed in the registration, re-selects C6 in 4 of 4 cells at the 85.2–98.2nd percentile of half B’s own landscape, against J ’s 9.3–82.4th. (ii) *Per-benchmark*, which is also what makes HumanEval+ the criterion’s best pool. The endpoint decomposes exactly by task, so the same held-out tasks re-partition by provenance at zero cost, recombining to the reported landscape maximum to 6 decimal places. On HumanEval+, which shares only the operator pool with the training split, C6 ranks at the 95.9 / 97.6 / 95.9 / 97.5th percentile, its best pool of the three, beating J in 4 of 4 cells and separating from it in a paired test in 3 of 4. Reading the same 428 tasks three ways adds no independent evidence for the overall comparison, but it does rule out the gain being an artefact of the benchmark the labels came from.

The sweep (Figure 4). $\lambda = n_{TP}/n_{FP}$ is read off the MBPP training split, so the question is whether it is a property of those labels or of selection endpoints generally. Twelve values of λ were scored in each of the four cells of Table 4. HumanEval+’s optimal plateau contains the training-split ratio in 4 of 4 cells, $\lambda=1$ lies strictly outside it every time, and $\lambda=0$, which ignores false alarms altogether, is never strictly better.

G Peer-dependent verdicts

A level-2 operator reads `ctx`, which is built from every raw sample for the task, so its verdict is a function of that peer electorate and not of the candidate alone. The two effects were measured apart.

the class weight is a property of the endpoint, not of the MBPP+ label ratio

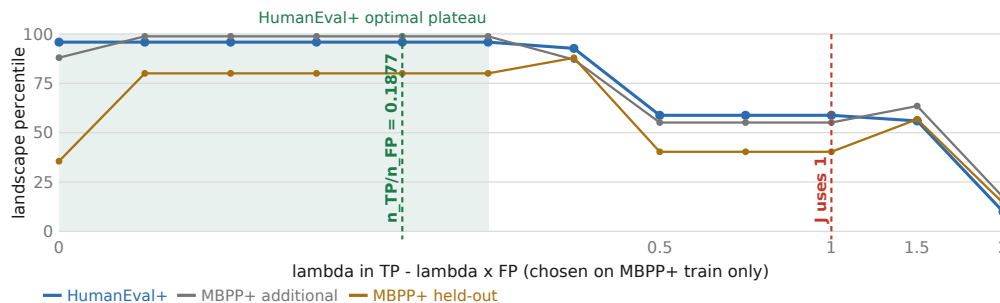


Figure 4: Sweeping λ in TP – λ FP, one line per held-out pool: HumanEval+’s optimal plateau (shaded) contains $\lambda=n_{TP}/n_{FP}=0.1877$, read off the MBPP+ training split, and excludes J ’s $\lambda=1$.

Against a frozen electorate, 0 of 35 operators change a single verdict, held-out or training, an exact identity we check rather than assume, and one every number here relies on. Against an electorate grown from 3424 to 12744 samples, all 35 change at least one, 1996 flags added and 440 removed, one operator more than doubling its own flag count on rows whose code never changed. So the wider interface buys discriminating power at the cost of referential stability, and a deployed level-2 operator has to pin its electorate the way we pin ours.

The direction of the change is what a conformity objection has to answer, and it is favourable. Every frozen selection gains $+0.0009$ to $+0.0036$ under the grown electorate with no material reordering, so the operators are reading signal out of peer context rather than tracking consensus for its own sake. On the same draw the five-operator rule of Section 5.3, selected on the training split and never re-selected, scores $+0.0220$ ($z=8.34$, $p=0.0010$, $+79/-12$ tasks) on 9320 candidates from three generators it was never fitted to. Raw deltas are not comparable across the two draws, because the deeper one is roomier (headroom 0.0995 against 0.0424), which is why both are quoted as a fraction of their own headroom: 22.1% against 18.9%. The operators are out of sample there; the task list is not.

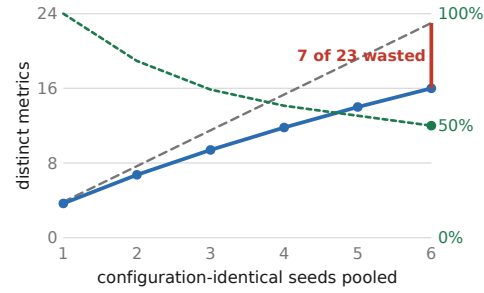
H Redundancy and the diversity veto

Figure 5 holds the redundancy evidence behind Section 6. The veto itself was run, not merely proposed: under the fold rotation of Section B it is at or below the plain union ($+0.0065/+0.0057/+0.0047$ at $m=2, 3, 4$, harming 9/11/11 tasks against the union’s 8), and development folds disable it in 5 of 9 directions. On the rediscovery-heavy pool of Section 6 the frozen rule holds on the 5 novel operators ($+0.0040$). It buys parsimony, 7 operators for 26 operators’ effect, rather than quality. It vetoes *flag-set* overlap at admission, which makes it a weaker instrument than the behavioural-novelty gate of Section 6, and the two fail differently.

I The judge and induction baselines

The judge, in four configurations. The judge sees the task statement, the prompt-visible asserts and the candidate source, which is exactly the level-1 interface `op(task, code)`, and never the hidden suite. Only the 1762 distinct (task, candidate) pairs that pass the visible checks are judged, because those are the only ones the endpoint scores. The configurations cross two models, the one that authors operators and the cheaper Claude Haiku 4.5, which is also one of the candidate generators of Section 3, so the cheap judge scores some of its own output; and two prompts: a one-word verdict, and a reason-then-verdict form whose wording states the training split’s own class balance, so the judge is given no statistic the operators are denied. Replies are cached by (pool, task, source hash) and a reply naming no verdict flags nothing, which is the conservative reading; rescoring the caches twice reproduces byte-identical output.

(a) waste, not exhaustion: the loop re-derives while classes remain



shared core: 8 of 24 classes span ≥ 2 schemes, holding 32 of 49 operators

(b) the indicated remedy blocks the duplicates without producing alternatives

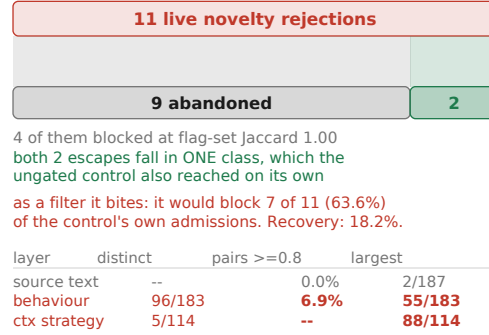


Figure 5: (a) Exact rarefaction over all 63 seed subsets and 720 arrival orders, not a bootstrap: the 23 operators admitted across those six seeds fall into 16 distinct behavioural classes, and the curve does not saturate. (b) The indicated remedy, run live: what became of each of the 11 candidates a behavioural-novelty term at admission blocks. The strip below reads redundancy at three layers, textual, behavioural and strategic, giving for each the distinct classes, the share of pairs at Jaccard ≥ 0.8 and the largest class.

Table 5: The judge as a metric, on the endpoint and null of Table 1. Precision, recall and accuracy read the judge as a *classifier* of the same pairs, which is a different question from the endpoint delta. The last two rows are the same two references as Table 1.

model	prompt	Δ	p	+/-	% headroom	precision	recall	accuracy
cheap	reason-then-verdict	+0.0070	0.0010	+21/-8	16.5	0.403	0.161	0.818
cheap	one word	+0.0044	0.0130	+20/-9	10.3	0.369	0.199	0.807
author	reason-then-verdict	+0.0040	0.0040	+14/-8	9.5	0.677	0.154	0.845
author	one word	+0.0036	0.0060	+17/-7	8.6	0.600	0.203	0.844
hand-written comparator		+0.0069	0.0130	+22/-10	16.3			
loop-authored operator		+0.0065	0.0010	+20/-6	15.4			

In Table 5, reasoning before answering raises the cheap model's delta and lowers its flag count, and both author-model configurations sit at roughly half that delta. As a classifier the author model is the more precise (0.677 against 0.403) and the less useful, which is the recall dominance of Section 3 again: a precise judge that says nothing on most bad candidates cannot change what is left to draw from. No reply in either reason-then-verdict configuration failed to name a verdict; the one-word configurations lost 1 of 1762 and 5 of 1762, and an unnamed verdict flags nothing.

Overlap and cost. Table 6 scores the two flag sets and their Boolean combinations through the same code. Read the overlap before the union delta: a union that beats both parts is only interesting when the parts disagree, and here they do.

Table 6: The loop-authored operator and the best judge configuration as flag sets, on the same endpoint. Jaccard between the two is 0.105: 27 flags in common of 257.

flag set	Δ	p	+/-	flags
operator only (set difference)	+0.0010	0.0679	+8/-6	99
judge only (set difference)	-0.0007	0.6563	+9/-9	131
both agree (intersection)	+0.0053	0.0010	+13/-1	27
either (union)	+0.0073	0.0010	+27/-12	257
operator	+0.0065	0.0010	+20/-6	126
judge	+0.0070	0.0010	+21/-8	158

Cost comes from the call ledgers rather than an estimate. The run that wrote the operator made 47 calls, and every candidate it has scored since cost nothing; all eight runs together made 376. The

judge costs one call per candidate with nothing amortised, so a single pass over this endpoint is 1762 calls, $37.5\times$ the run that produced the operator or $4.7\times$ all eight runs including the two that admitted nothing, and every future candidate costs that again.

Verifier induction. Same author model, same system prompts, same four screens in the same order, same training split, endpoint and null. Two things change deliberately: candidates are commissioned against a balanced random labelled sample rather than against a signature collision, and they are admitted for predicting the *label*, at least 3 true positives at precision at least 0.5, which are the label-space analogues of the deployed gate’s three helped tasks and helped $>$ hurt. Table 7 reports both interfaces under three ways of combining whatever was admitted.

Table 7: Verifier induction at each interface. Combiners are the plain union of admitted checks, a ≥ 2 -of- k vote, and the single check with the best *training* F_1 ; only the last is a rule a practitioner could apply without held-out labels.

	level 1, <code>detect(task, code)</code>	level 2, <code>detect(task, code, ctx)</code>
candidates commissioned	2 runs $\times$ 24	1 run $\times$ 16
admitted	2	6
union of the admitted checks	+0.0016 ($p=0.19$)	+0.0053 ($p=0.0120$)
≥ 2 -of- k vote	+0.0000 ($p=0.52$)	+0.0053 ($p=0.0010$)
best single by training F_1	+0.0016 ($p=0.19$)	+0.0054 ($p=0.0060$)

Level 1 fails the way our own level-1 search fails: 48 calls, 2 admissions, one run admitting nothing, and no combination separating from its null. Level 2 works. The screens fired on this baseline as they do on our own author path, rejecting candidates for source-pattern fitting and for prompt dispatch, so it was given no exemption our method does not have. Its best *single* check reaches +0.0074, above both the comparator and our operator, but that check is chosen by reading held-out deltas and is not a legitimate headline. The legitimate reading is the warning: on this endpoint the ceiling reachable by predictive-fit admission at the wider interface is at least as high as the one we report.

Label prediction as a composition criterion. The same criterion swap can be made with no model and no sampling at all, because Section 5.3’s enumeration puts every subset in memory, so a criterion is a total order over a finite set and its choice, delta and percentile are counts rather than estimates. Carrying J as the instrument check, its argmax reproduces the published one in all four cells.

Table 8: Held-out Δ and exact landscape percentile of four criteria across the four cells of Table 4. C8 is precision under the same floor of 3 true positives, and it is degenerate with the floor in place: it picks the same two-operator set flagging ten tasks in every cell.

criterion	26 ops, full	35 ops, full	26 ops, frozen	35 ops, frozen
C1, $J = TP - FP$	+0.0155 (56.3)	+0.0140 (42.3)	+0.0220 (85.7)	+0.0079 (4.9)
C6, balanced accuracy	+0.0286 (98.7)	+0.0253 (91.1)	+0.0236 (92.7)	+0.0236 (94.1)
C7, label-prediction F_1	+0.0244 (89.2)	+0.0231 (85.7)	+0.0219 (85.0)	+0.0218 (85.8)
C8, precision	+0.0038 (0.0)	+0.0038 (0.0)	+0.0038 (0.3)	+0.0038 (0.1)

Table 8 adds two criteria to the six of Table 4, and one of them wins against J : C7 beats it in three cells, ties it in the fourth, and does not collapse when the pool grows from 26 operators to 35, which J does. So the gap Section 5.3 measures is partly recoverable by the criterion the related work already uses, and C6 recovers more of it in all four cells. C7 selects a different subset from J each time, at Jaccard 0.22 to 0.67 against J ’s choice, so this is not two names for one ranking.